



GUIA PARA O GRUPO DE ALUNOS

Como funciona o MVC neste sistema

Este texto explica o padrão Model–View–Controller usado no Clube de Mahjong, onde cada arquivo mora no projeto e como continuar a programar sem misturar as camadas.

Clube de Mahjong
PHP · JavaScript · CSS · SQLite

Servidor local: `php -S localhost:8081 -t public`
Acesso inicial: `admin / admin`

SUMÁRIO

O que você vai encontrar

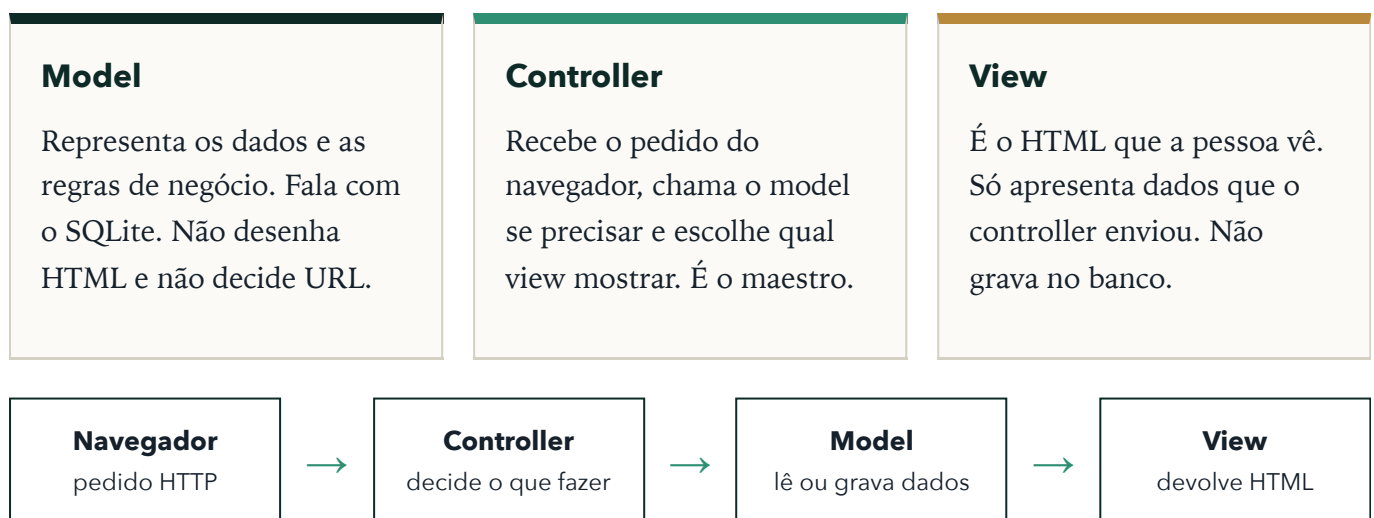
1. O que é MVC
2. Como isso aparece neste projeto
3. Onde estão os arquivos
4. Ciclo de um pedido HTTP
5. O dashboard (modelo a copiar)
6. Exemplo: tela de login
7. Como adicionar uma tela nova
8. Regras básicas
9. Como subir o sistema

O sistema não é um jogo de mahjong. É um programa para administrar um **clube**: pessoas, eventos, partidas e progresso. Várias telas ainda são placeholders — o grupo vai preenchê-las.

CONCEITO

1. O que é MVC

MVC é uma forma de organizar o código em três papéis. Cada um faz uma coisa só. Assim, um aluno pode mexer na tela sem quebrar o banco, e outro pode mexer nos dados sem redesenhar o HTML.



A analogia da cozinha: o *controller* é o garçom (anota o pedido), o *model* é a cozinha (prepara o prato), a *view* é o prato na mesa (o que o cliente vê).

MAPEAMENTO

2. Como isso aparece neste projeto

Não usamos Laravel. É PHP vanilla, com pastas no lugar das camadas. O arquivo `public/index.php` só liga URL → método do controller. Ele não deve crescer com regra de negócio.

Papel MVC	Pasta / arquivo	Exemplos neste sistema
Entrada	<code>public/index.php</code>	Rotas: <code>/login</code> , <code>/</code> , <code>/eventos</code> ...
Controller	<code>src/controllers/</code>	<code>AuthController</code> , <code>DashboardController</code> , <code>EventController</code>
Model	<code>src/models/</code>	<code>User</code> , <code>Member</code> , <code>Event</code> , <code>GameMatch</code> , <code>ProgressNote</code>
View	<code>views/</code>	<code>auth/login.php</code> , <code>dashboard.php</code> , <code>events/index.php</code>
Aparência	<code>public/assets/css/</code>	<code>app.css</code> (geral), <code>login.css</code> (cores do login)
Comportamento no browser	<code>public/assets/js/app.js</code>	Menu no celular, avisos, formulário ocupado
Banco	<code>database/</code> e <code>data/</code>	<code>schema.sql</code> , <code>seed.sql</code> , <code>app.sqlite</code>

Há ainda peças de apoio que não são M, V nem C, mas sustentam o MVC:

Arquivo	Para que serve
<code>src/bootstrap.php</code>	Sessão, autoload das classes, helpers (<code>view()</code> , CSRF, flash).
<code>src/db.php</code>	Abre o SQLite e aplica schema/seed na primeira execução.
<code>src/Router.php</code>	Encaixa o caminho da URL no controller certo. Se não achar, chama o 404.
<code>src/AuthService.php</code>	Sessão: quem está logado, entrar, sair. Não desenha tela.
<code>src/controllers/Controller.php</code>	Classe base dos controllers (<code>requireAuth()</code> , <code>view()</code>).
<code>config.php</code>	Nome do clube e caminhos do banco.

A classe da partida se chama `GameMatch` (e não `Match`) porque `match` é palavra reservada do PHP 8.

MAPA

3. Onde estão os arquivos

```
mahjong/  
public/      ← o navegador só "vê" esta pasta  
  index.php  ← rotas (front controller)  
  assets/css/app.css  
  assets/css/login.css  
  assets/js/app.js  
src/  
  bootstrap.php  
  db.php  
  Router.php  
  AuthService.php  
  controllers/ ← CONTROLLER  
    Controller.php  
    AuthController.php  
    DashboardController.php  
    EventController.php  
    MatchController.php  
    MemberController.php  
    RankingController.php  
    PageController.php  
    ErrorController.php  
  models/     ← MODEL  
    User.php  
    Member.php  
    Event.php  
    GameMatch.php  
    ProgressNote.php  
  views/      ← VIEW  
    layout.php ← casca (sidebar ou tela de visitante)  
    dashboard.php  
    coming-soon.php  
    auth/login.php  
    auth/register.php  
    events/index.php  
    matches/index.php  
    members/index.php  
    progress/index.php  
    errors/404.php  
  database/schema.sql  
  database/seed.sql  
  data/app.sqlite ← gerado sozinho; não vai para o Git  
  documentacao/mvc/ ← este guia  
  Imagens_base/   ← capturas de telas planejadas
```

PASSO A PASSO

4. Ciclo de um pedido HTTP

Depois do login, a tela inicial é o **dashboard** (URL `/`), não a de eventos. Eventos, partidas e o restante são telas seguintes, abertas pelos cards ou pelo menu.

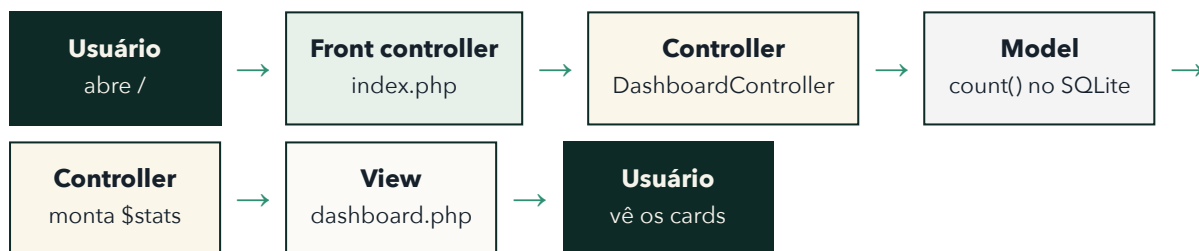
1 O aluno, já autenticado, abre `http://localhost:8081/`. O document root é `public/`.

- 2 `public/index.php` carrega o bootstrap e registra: `$router->get('/', [$dashboard, 'index'])`.
- 3 O Router chama `DashboardController::index()`.
- 4 O controller exige login (`requireAuth()`). Se não houver sessão, redireciona para `/login`.
- 5 O controller pede dados aos models: `Member::count()`, `Event::count()`, `GameMatch::count()`, `ProgressNote::count()`.
- 6 De volta ao controller, ele chama `$this->view('dashboard', ...)`. O HTML de `views/dashboard.php` entra no `views/layout.php`.
- 7 O navegador mostra os cards. Só então, se a pessoa clicar em Eventos, começa um ciclo novo em `/eventos`.

MODELO A COPIAR

5. O dashboard, peça por peça

O dashboard já está completo no padrão MVC. As outras funções (torneios, listagem de eventos, partidas, etc.) devem repetir este mesmo caminho:



O model *não* fala com a view. O controller pede os dados, recebe os números e só então chama a view. Por isso o caminho volta ao controller no meio.

O que cada arquivo faz nesse pedido GET /

Ordem	Papel	Arquivo	Responsabilidade
1	Usuário	Navegador	Já autenticado, pede <code>http://localhost:8081/</code> (GET).
2	Front controller	<code>public/index.php</code>	Carrega o bootstrap, cria <code>\$dashboard = new DashboardController()</code> e registra <code>\$router->get('/', [\$dashboard, 'index'])</code> . O Router despacha para esse método.

Ordem	Papel	Arquivo	Responsabilidade
3	Controller	src/controllers/DashboardController.php	index() chama requireAuth() (sem sessão, redireciona para /login) e pede totais aos models.
4	Model	Member , Event , GameMatch , ProgressNote	Cada um executa COUNT(*) na sua tabela via db() . Não conhecem HTML nem a URL.
5	Controller de novo	mesmo DashboardController	Junta os retornos em \$stats e chama \$this->view('dashboard', ...) . A view só recebe arrays prontos.
6	View	views/dashboard.php + views/layout.php	Imprime o nome e os quatro cards. O layout coloca a barra lateral. CSS em app.css .
7	Usuário	Navegador	Recebe o HTML. Clicar num card dispara um novo ciclo MVC (por exemplo GET /eventos).

Código real do controller

```
final class DashboardController extends Controller
{
    public function index(): void
    {
        $user = $this->requireAuth();    // sessão
        $this->view('dashboard', [
            'title' => 'Dashboard',
            'user' => $user,
            'currentPath' => '/',
            'stats' => [
                'members' => Member::count(),    // model
                'events' => Event::count(),    // model
                'matches' => GameMatch::count(), // model
                'progress' => ProgressNote::count(),
            ],
        ]);
    }
}
```

Código real de um model (Event)

```
final class Event
{
    public static function count(): int
    {
        return (int) db()->query('SELECT COUNT(*) FROM events')->fetchColumn();
    }
}
```

Os outros três models do dashboard (`Member` , `GameMatch` , `ProgressNote`) são iguais: só mudam a tabela. A view usa `$stats['events']` etc. — nunca chama SQL.

Para implementar **Torneios** (hoje só um placeholder em `PageController::torneios()`), copie este desenho: rota em `index.php` → `TorneioController` → `Torneio` (model) → de volta ao controller → `views/torneios/index.php` → usuário. Não coloquem SQL na view e não desenhem HTML no model.

CASO CONCRETO

6. Exemplo: tela de login

Camada	Arquivo	O que faz no login
Rota GET	<code>public/index.php</code>	<code>/login</code> → <code>AuthController::showLogin()</code>
Controller	<code>src/controllers/AuthController.php</code>	Se já está logado, manda para o dashboard. Senão, abre a view.
View	<code>views/auth/login.php</code>	Formulário. Sem cores misturadas no PHP.
CSS	<code>public/assets/css/login.css</code>	Variáveis <code>--login*</code> no topo: é aqui que se mudam as cores do login.
Rota POST	<code>AuthController::login()</code>	Valida CSRF, lê usuário e senha, chama o serviço de autenticação.
Model + sessão	<code>User</code> e <code>AuthService</code>	<code>User</code> busca no SQLite; <code>AuthService</code> grava <code>\$_SESSION['user_id']</code> .

Usuário e senha iniciais de desenvolvimento: **admin** / **admin** (definidos em `database/seed.sql`).

PARA O GRUPO

7. Como adicionar uma tela nova

Repitam o dashboard. Imagine listar eventos de verdade, não só o placeholder. Siga esta ordem — não comece pelo HTML solto.

- 1 Model.** Em `src/models/Event.php`, crie métodos como `all()` ou `create()` com SQL via `db()` (PDO, prepared statements).
- 2 Controller.** Em `EventController`, um método `index()` busca os eventos no model e passa o array para a view. Um `store()` trata o POST do formulário.
- 3 Rota.** Em `public/index.php`: `$router->get('/eventos', [$events, 'index']);` e, se houver formulário, um `post`.
- 4 View.** Atualize `views/events/index.php` só com HTML e `<pre>` para imprimir dados já preparados. Use `e()` para escapar texto.
- 5 CSS.** Classes novas em `public/assets/css/app.css` (ou um CSS da página, como o login). Nada de `style="..."` no PHP.

Páginas ainda vazias (torneios, notícias, usuários, configurações) passam por `PageController` e `views/coming-soon.php`. Quando forem implementadas, criem controller + model próprios e troquem a rota.

COMBINADO

8. Regras básicas

1. SQL fica no model. HTML fica na view. O controller só liga os dois.
2. CSS em `public/assets/css/`. JavaScript em `public/assets/js/`. Sem `style` nem `script` no PHP das telas.
3. Formulário POST leva `csrfField()`. Consulta no banco usa PDO com parâmetro (`:nome`), não concatena string.
4. Senha grava com `password_hash`. Nunca em texto puro.
5. Tabela nova ou coluna nova: alterar `database/schema.sql` e apagar `data/app.sqlite` para recriar o banco.

9. Como subir o sistema

Na raiz do projeto (a porta **8081** foi escolhida porque a 8080 já está em outro programa):

```
php -S localhost:8081 -t public
```

Abra `http://localhost:8081`. Entre com `admin` / `admin`. Depois de clonar o repositório, o SQLite é criado sozinho em `data/app.sqlite` na primeira execução.

Lista curta de cada arquivo de código: `DOCUMENTACAO.md` na raiz. Este PDF explica o *porquê* da arquitetura; aquele arquivo é o inventário.